Problem Statement

To estimate the delivery time for food for it customers on the basis of what they are ordering, from where they are ordering and also who are the delivery partners.

Feature Details

market_id : integer id for the market where the restaurant lies

created_at : the timestamp at which the order was placed

actual_delivery_time : the timestamp when the order was delivered

store_primary_category : category for the restaurant

order_protocol : integer code value for order protocol(how the order was placed ie: through porter, call to restaurant, pre booked, third part etc)

total_items subtotal : final price of the order

num_distinct_items : the number of distinct items in the order

min_item_price : price of the cheapest item in the order

max_item_price : price of the costliest item in order

total_onshift_partners : number of delivery partners on duty at the time order was placed

total_busy_partners : number of delivery partners attending to other tasks

total_outstanding_orders : total number of orders to be fulfilled at the moment

estimated_store_to_consumer_driving_duration : approximate travel time from restaurant to customer

Loading Of Libraries

In [1]:
```
!pip install keras-tuner
```

```
Collecting keras-tuner
  Downloading keras_tuner-1.4.8-py3-none-any.whl.metadata (5.6 kB)
Requirement already satisfied: keras in /usr/local/lib/python3.12/dist-packages
(from keras-tuner) (3.10.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-pack
ages (from keras-tuner) (26.0)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packa
ges (from keras-tuner) (2.32.4)
Collecting kt-legacy (from keras-tuner)
  Downloading kt_legacy-1.0.5-py3-none-any.whl.metadata (221 bytes)
Requirement already satisfied: grpcio in /usr/local/lib/python3.12/dist-package
s (from keras-tuner) (1.78.0)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packa
ges (from keras-tuner) (5.29.6)
Requirement already satisfied: typing-extensions~=4.12 in /usr/local/lib/python
3.12/dist-packages (from grpcio->keras-tuner) (4.15.0)
Requirement already satisfied: absl-py in /usr/local/lib/python3.12/dist-packag
es (from keras->keras-tuner) (1.4.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages
(from keras->keras-tuner) (2.0.2)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages
(from keras->keras-tuner) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages
(from keras->keras-tuner) (0.1.0)
Requirement already satisfied: h5py in /usr/local/lib/python3.12/dist-packages
(from keras->keras-tuner) (3.15.1)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-package
s (from keras->keras-tuner) (0.19.0)
Requirement already satisfied: ml-dtypes in /usr/local/lib/python3.12/dist-pack
ages (from keras->keras-tuner) (0.5.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/pytho
n3.12/dist-packages (from requests->keras-tuner) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-p
ackages (from requests->keras-tuner) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/
dist-packages (from requests->keras-tuner) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/
dist-packages (from requests->keras-tuner) (2026.2.25)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python
3.12/dist-packages (from rich->keras->keras-tuner) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python
3.12/dist-packages (from rich->keras->keras-tuner) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-pac
kages (from markdown-it-py>=2.2.0->rich->keras->keras-tuner) (0.1.2)
Downloading keras_tuner-1.4.8-py3-none-any.whl (129 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 129.4/129.4 kB 5.4 MB/s eta 0:00:00
Downloading kt_legacy-1.0.5-py3-none-any.whl (9.6 kB)
Installing collected packages: kt-legacy, keras-tuner
Successfully installed keras-tuner-1.4.8 kt-legacy-1.0.5
```

In [140…

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# importing keras Libraries

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

from tensorflow.keras.layers import BatchNormalization,Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.callbacks import ReduceLROnPlateau
from tensorflow.keras.layers import (
    Input,
    Embedding,
    Flatten,
    Dense,
    Concatenate,
    BatchNormalization,
    Dropout
)

from tensorflow.keras.models import Model
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score


from keras_tuner import RandomSearch
```

In [3]:
```
df= pd.read_csv("data_2.csv")
```

In [4]:
```
df.head()
```

Out[4]:

	market_id	created_at	actual_delivery_time	store_primary_category	order_pr
0	1.0	2015-02-06 22:24:17	2015-02-06 23:11:17	4	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:33:25	46	
2	2.0	2015-02-16 00:11:35	2015-02-16 01:06:35	36	
3	1.0	2015-02-12 03:36:46	2015-02-12 04:35:46	38	
4	1.0	2015-01-27 02:12:36	2015-01-27 02:58:36	38	

In [5]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175777 entries, 0 to 175776
Data columns (total 14 columns):
 #   Column                                        Non-Null Count   Dtype
---  ------                                        --------------   -----
 0   market_id                                     175777 non-null  float64
 1   created_at                                    175777 non-null  object
 2   actual_delivery_time                          175777 non-null  object
 3   store_primary_category                        175777 non-null  int64
 4   order_protocol                                175777 non-null  float64
 5   total_items                                   175777 non-null  int64
 6   subtotal                                      175777 non-null  int64
 7   num_distinct_items                            175777 non-null  int64
 8   min_item_price                                175777 non-null  int64
 9   max_item_price                                175777 non-null  int64
 10  total_onshift_dashers                         175777 non-null  float64
 11  total_busy_dashers                            175777 non-null  float64
 12  total_outstanding_orders                      175777 non-null  float64
 13  estimated_store_to_consumer_driving_duration  175777 non-null  float64
dtypes: float64(6), int64(6), object(2)
memory usage: 18.8+ MB
```

In [6]:
```
df.isnull().sum()*100/len(df)
```

Out[6]:

	0
market_id	0.0
created_at	0.0
actual_delivery_time	0.0
store_primary_category	0.0
order_protocol	0.0
total_items	0.0
subtotal	0.0
num_distinct_items	0.0
min_item_price	0.0
max_item_price	0.0
total_onshift_dashers	0.0
total_busy_dashers	0.0
total_outstanding_orders	0.0
estimated_store_to_consumer_driving_duration	0.0

dtype: float64

We do not have any NULL or missing values in the dataset

In [7]:
```
df.describe()
```

Out[7]:

	market_id	store_primary_category	order_protocol	total_items	
count	175777.000000	175777.000000	175777.000000	175777.000000	17
mean	2.743726	35.887949	2.911752	3.204976	
std	1.330963	20.728254	1.513128	2.674055	
min	1.000000	0.000000	1.000000	1.000000	
25%	2.000000	18.000000	1.000000	2.000000	
50%	2.000000	38.000000	3.000000	3.000000	
75%	4.000000	55.000000	4.000000	4.000000	
max	6.000000	72.000000	7.000000	411.000000	2

In [8]:
```
df.nunique()
```

Out[8]:

	0
market_id	6
created_at	162649
actual_delivery_time	160344
store_primary_category	73
order_protocol	7
total_items	54
subtotal	8182
num_distinct_items	20
min_item_price	2251
max_item_price	2585
total_onshift_dashers	172
total_busy_dashers	158
total_outstanding_orders	281
estimated_store_to_consumer_driving_duration	1318

dtype: int64

Data Pre Processing

In [89]:
```
df_new= df.copy()
```

In [90]:
```
df_new.head()
```

Out[90]:

	market_id	created_at	actual_delivery_time	store_primary_category	order_pr
0	1.0	2015-02-06 22:24:17	2015-02-06 23:11:17	4	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:33:25	46	
2	2.0	2015-02-16 00:11:35	2015-02-16 01:06:35	36	
3	1.0	2015-02-12 03:36:46	2015-02-12 04:35:46	38	
4	1.0	2015-01-27 02:12:36	2015-01-27 02:58:36	38	

In [91]:
```
df_new["created_at"] = pd.to_datetime(df_new["created_at"])
df_new["actual_delivery_time"] = pd.to_datetime(df_new["actual_delivery_time"]
```

Feature engineering

In [92]:
```
df_new["Time_taken_delivery"]= df_new["actual_delivery_time"] - df_new["create
```

Time based Features

In [93]:
```
df_new["order_hour"] = df_new["created_at"].dt.hour
df_new["order_day_of_week"] = df_new["created_at"].dt.weekday
df_new["is_weekend"] = df_new["order_day_of_week"].isin([5,6]).astype(int)
df_new["delivery_in_minutes"] = df_new["Time_taken_delivery"].dt.total_seconds
```

Here , **delivery_in_minutes** is our **target Variable**

Feature to estimate demand

In [94]:
```
df_new["orders_per_dasher"] = np.round(
    np.where(
        df_new["total_onshift_dashers"] > 0,
        df_new["total_outstanding_orders"] / df_new["total_onshift_dashers"],
        0
    ),
```

```
2)

df_new["busy_dasher_ratio"] = np.round(
    np.where(
        df_new["total_onshift_dashers"] > 0,
        df_new["total_busy_dashers"] / df_new["total_onshift_dashers"],
        0
    ),
2)
```

Price-related Features

In [95]:
```
df_new["price_range"] = df_new["max_item_price"] - df_new["min_item_price"]
df_new["avg_item_price"] = df_new["subtotal"] / df_new["total_items"]
```

In [35]:
```
df_new.head()
```

Out[35]:

	market_id	created_at	actual_delivery_time	store_primary_category	order_pr
0	1.0	2015-02-06 22:24:17	2015-02-06 23:11:17	4	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:33:25	46	
2	2.0	2015-02-16 00:11:35	2015-02-16 01:06:35	36	
3	1.0	2015-02-12 03:36:46	2015-02-12 04:35:46	38	
4	1.0	2015-01-27 02:12:36	2015-01-27 02:58:36	38	

5 rows × 23 columns

In [96]:
```
df_new.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175777 entries, 0 to 175776
Data columns (total 23 columns):
 #   Column                                        Non-Null Count   Dtype
---  ------                                        --------------   -----
 0   market_id                                     175777 non-null  float64
 1   created_at                                    175777 non-null  datetime6
4[ns]
 2   actual_delivery_time                          175777 non-null  datetime6
4[ns]
 3   store_primary_category                        175777 non-null  int64
 4   order_protocol                                175777 non-null  float64
 5   total_items                                   175777 non-null  int64
 6   subtotal                                      175777 non-null  int64
 7   num_distinct_items                            175777 non-null  int64
 8   min_item_price                                175777 non-null  int64
 9   max_item_price                                175777 non-null  int64
 10  total_onshift_dashers                         175777 non-null  float64
 11  total_busy_dashers                            175777 non-null  float64
 12  total_outstanding_orders                      175777 non-null  float64
 13  estimated_store_to_consumer_driving_duration  175777 non-null  float64
 14  Time_taken_delivery                           175777 non-null  timedelta6
4[ns]
 15  order_hour                                    175777 non-null  int32
 16  order_day_of_week                             175777 non-null  int32
 17  is_weekend                                    175777 non-null  int64
 18  delivery_in_minutes                           175777 non-null  float64
 19  orders_per_dasher                             175777 non-null  float64
 20  busy_dasher_ratio                             175777 non-null  float64
 21  price_range                                   175777 non-null  int64
 22  avg_item_price                                175777 non-null  float64
dtypes: datetime64[ns](2), float64(10), int32(2), int64(8), timedelta64[ns](1)
memory usage: 29.5 MB
```

Outlier detection

In [97]:
```
df_new.skew(numeric_only=True)
```

Out[97]:

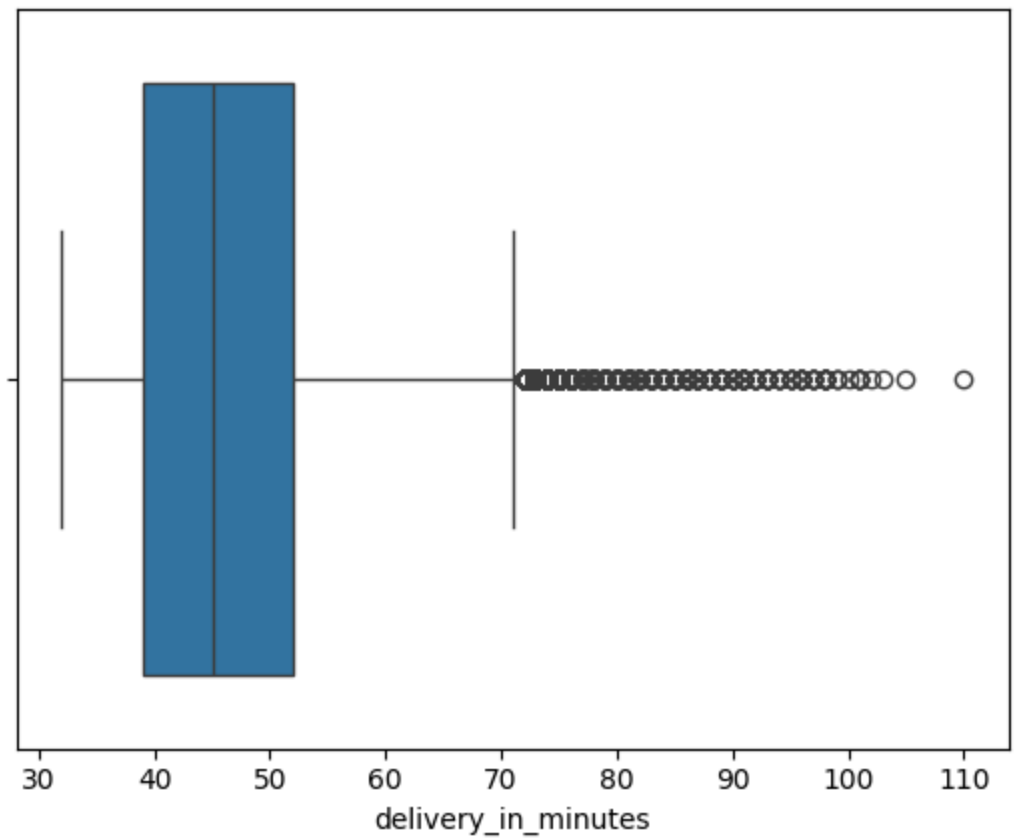
	0
market_id	0.225556
store_primary_category	-0.096707
order_protocol	0.109423
total_items	23.286019
subtotal	1.918445
num_distinct_items	1.574312
min_item_price	2.338844
max_item_price	2.204340
total_onshift_dashers	0.856865
total_busy_dashers	0.778571
total_outstanding_orders	1.191167
estimated_store_to_consumer_driving_duration	0.139994
order_hour	0.628818
order_day_of_week	-0.173047
is_weekend	0.649275
delivery_in_minutes	0.762191
orders_per_dasher	10.434772
busy_dasher_ratio	17.548935
price_range	2.023874
avg_item_price	2.385706

dtype: float64

|skewness| > 1 → strong skew

In [72]:
```
sns.boxplot(x=df_new["delivery_in_minutes"])
```

Out[72]:
```
<Axes: xlabel='delivery_in_minutes'>
```



Clipping extreme values (outliers)

In [98]:
```
df_new["delivery_in_minutes"] = df_new["delivery_in_minutes"].clip(
    lower=df_new["delivery_in_minutes"].quantile(0.01),
    upper=df_new["delivery_in_minutes"].quantile(0.99)
)
df_new["orders_per_dasher"] = df_new["orders_per_dasher"].clip(
    lower=df_new["orders_per_dasher"].quantile(0.01),
    upper=df_new["orders_per_dasher"].quantile(0.99)
)
df_new["busy_dasher_ratio"] = df_new["busy_dasher_ratio"].clip(
    lower=df_new["busy_dasher_ratio"].quantile(0.01),
    upper=df_new["busy_dasher_ratio"].quantile(0.99)
)
```

In [99]:
```
df_new.skew(numeric_only=True)
```

Out[99]:

	0
market_id	0.225556
store_primary_category	-0.096707
order_protocol	0.109423
total_items	23.286019
subtotal	1.918445
num_distinct_items	1.574312
min_item_price	2.338844
max_item_price	2.204340
total_onshift_dashers	0.856865
total_busy_dashers	0.778571
total_outstanding_orders	1.191167
estimated_store_to_consumer_driving_duration	0.139994
order_hour	0.628818
order_day_of_week	-0.173047
is_weekend	0.649275
delivery_in_minutes	0.588655
orders_per_dasher	-0.171388
busy_dasher_ratio	0.023543
price_range	2.023874
avg_item_price	2.385706

dtype: float64

EDA Univariate and Bivariate Analysis

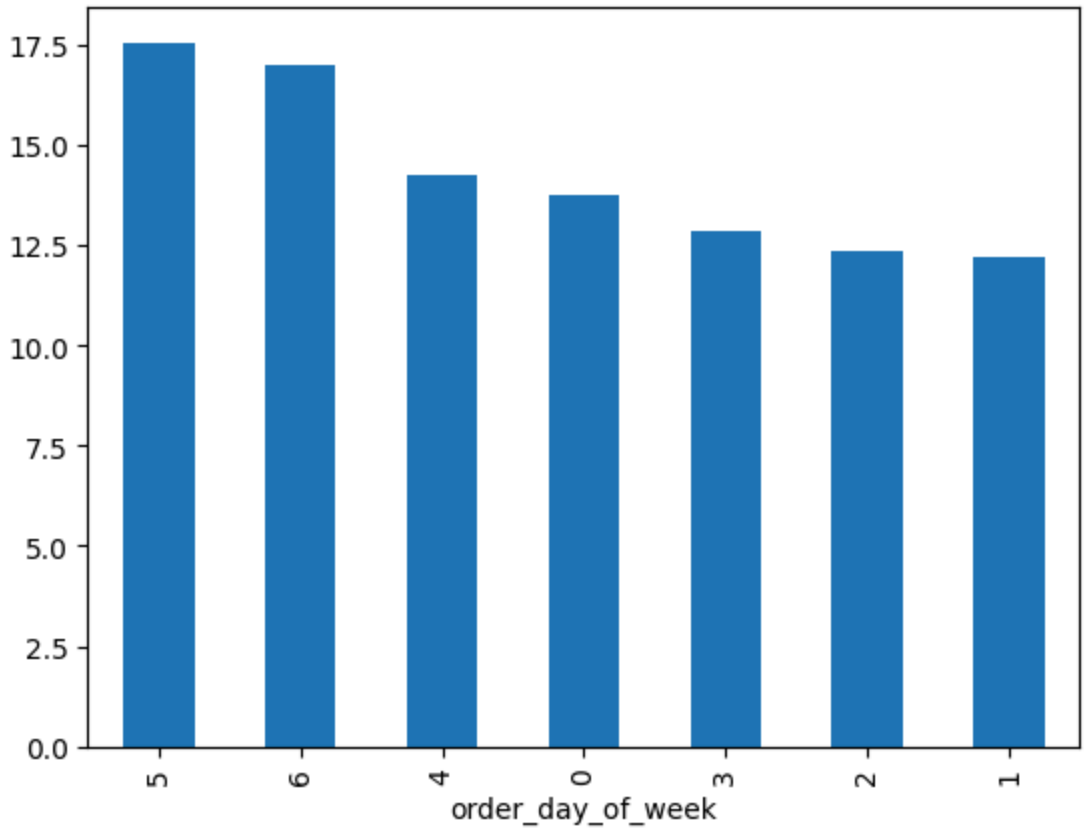
In [100…

```
day_percent = df_new["order_day_of_week"].value_counts(normalize=True) * 100
day_percent = day_percent.sort_values(ascending=False)

day_percent.plot(kind="bar")
```

Out[100…

```
<Axes: xlabel='order_day_of_week'>
```



We can see that most number of orders are being delivered on the weekends

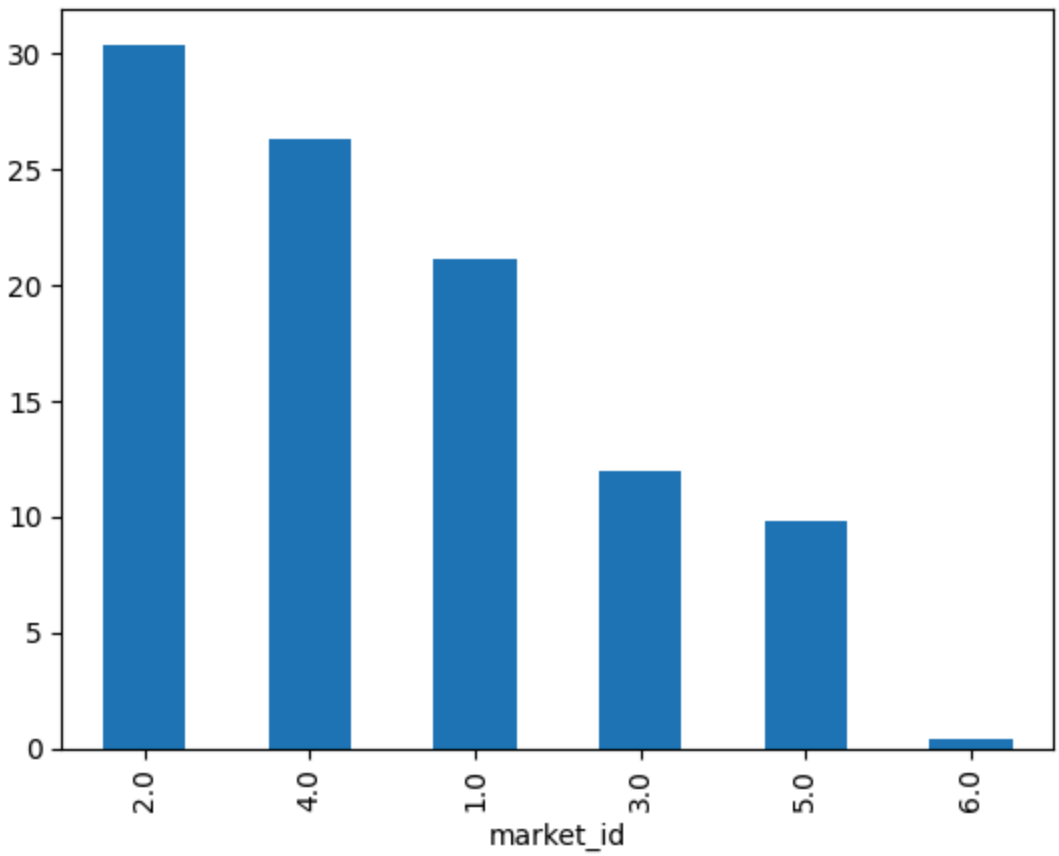
In [101…

```
market_orders = df_new["market_id"].value_counts(normalize=True) * 100
market_orders = market_orders.sort_values(ascending=False)

market_orders.plot(kind="bar")
```

Out[101…

```
<Axes: xlabel='market_id'>
```



Most of the orders are from market 2 ID

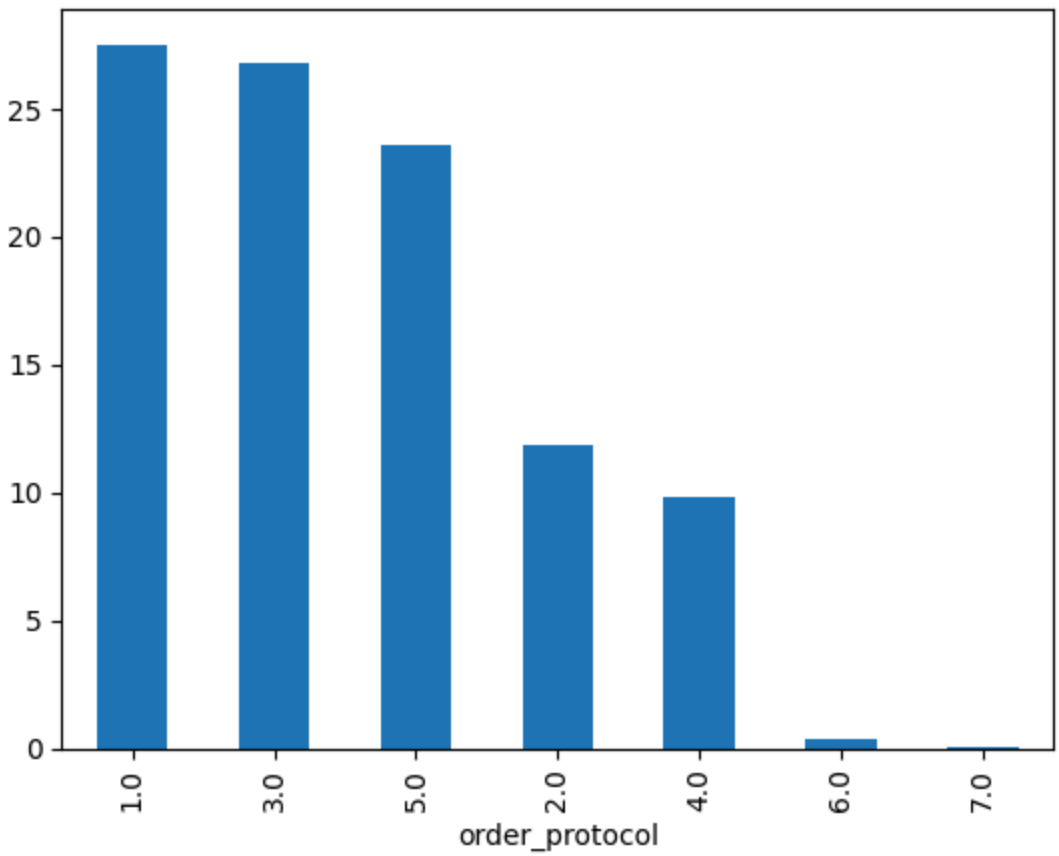
In [102…
```
df_new["order_protocol"].unique()
```

Out[102…
```
array([1., 2., 3., 5., 4., 6., 7.])
```

In [103…
```
order_protocol = df_new["order_protocol"].value_counts(normalize=True) * 100
order_protocol = order_protocol.sort_values(ascending=False)

order_protocol.plot(kind="bar")
```

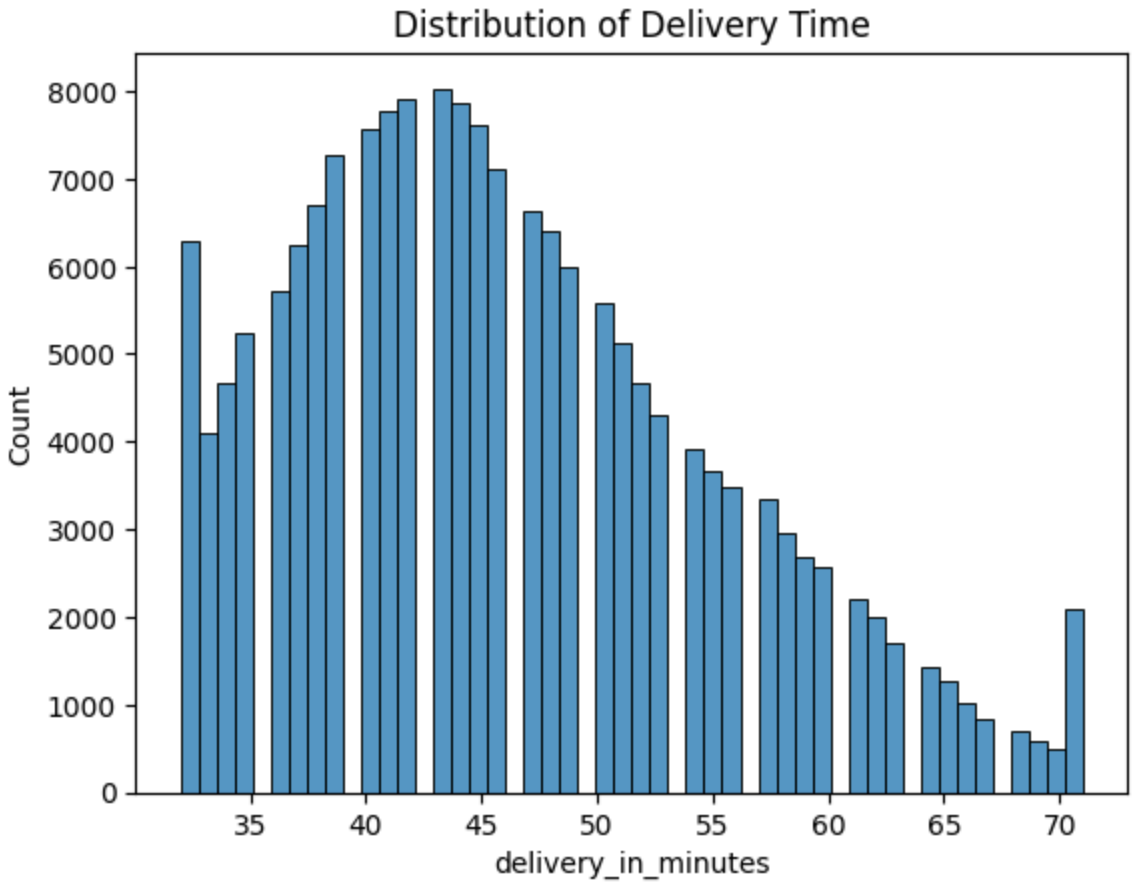
Out[103…
```
<Axes: xlabel='order_protocol'>
```



1,3,5 protocols were the most used source for ordering food

In [104…

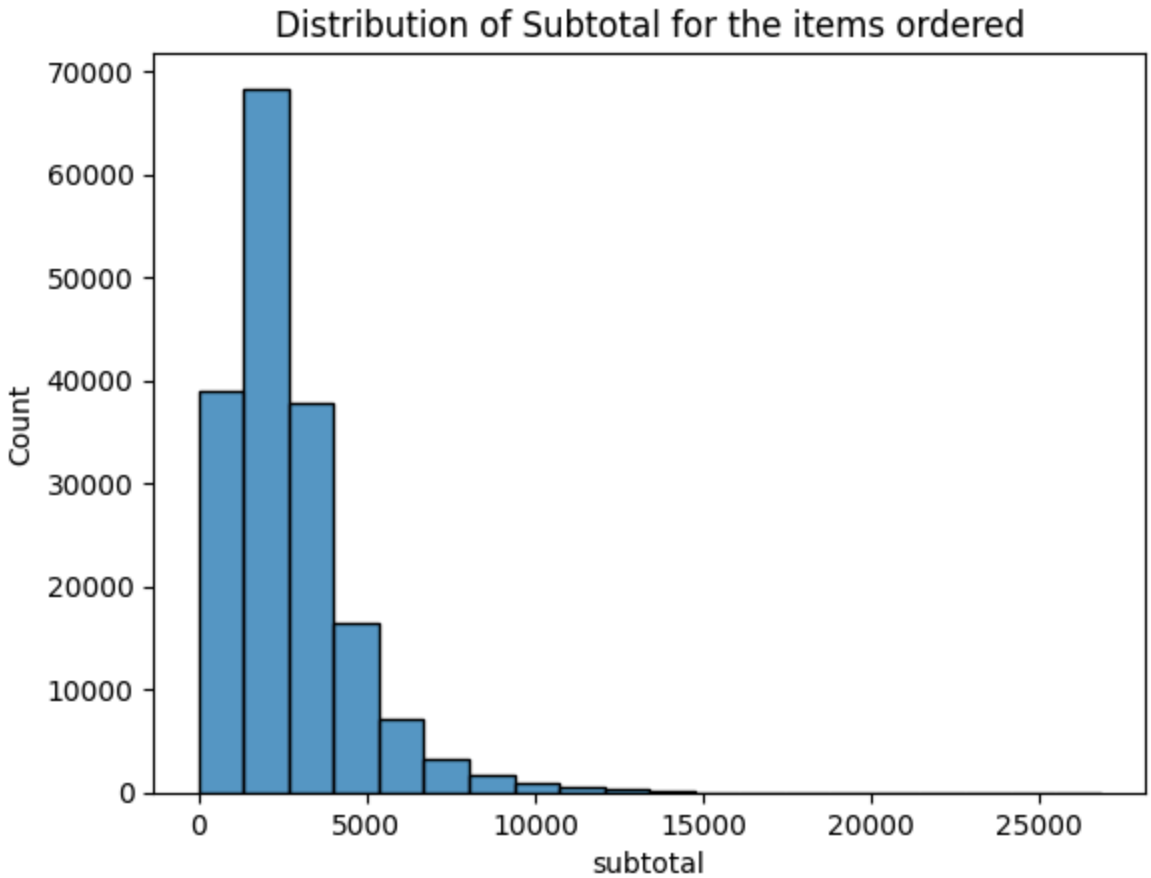
```
sns.histplot(df_new["delivery_in_minutes"], bins=50)
plt.title("Distribution of Delivery Time")
plt.show()
```



Most of the deliveries lies between 40 to 50 mins. we can observe that there are extreme delays as well for delivery time > 1 hour

In [105…

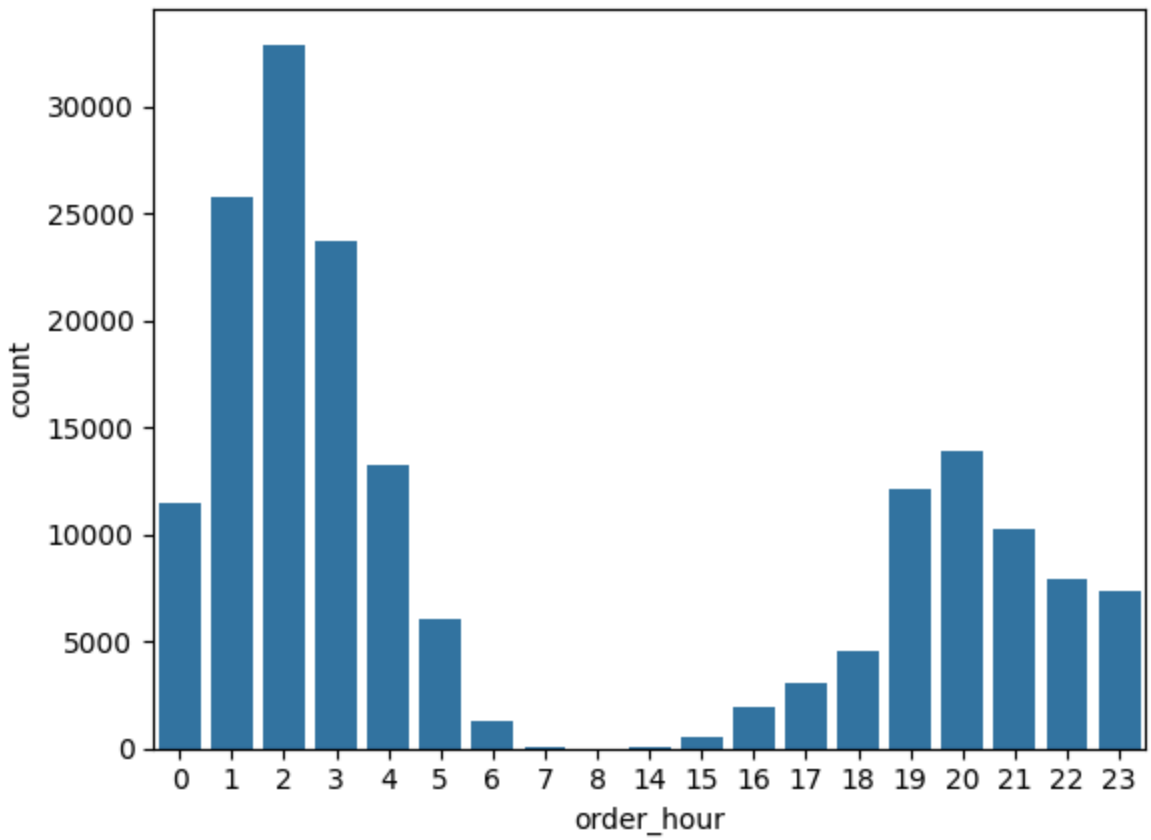
```
sns.histplot(df_new["subtotal"], bins=20)
plt.title("Distribution of Subtotal for the items ordered")
plt.show()
```



we can see a right skewed distribution graph with mots of the orders have total order value < 5000 units

In [106…
```
sns.countplot(x="order_hour", data=df_new)
```

Out[106…
```
<Axes: xlabel='order_hour', ylabel='count'>
```



Most of the food orders are being placed from 12 am in the night till 3 am in the morning. also we can see a spike in orders during the evning time as well from 4 pm till 11 pm.

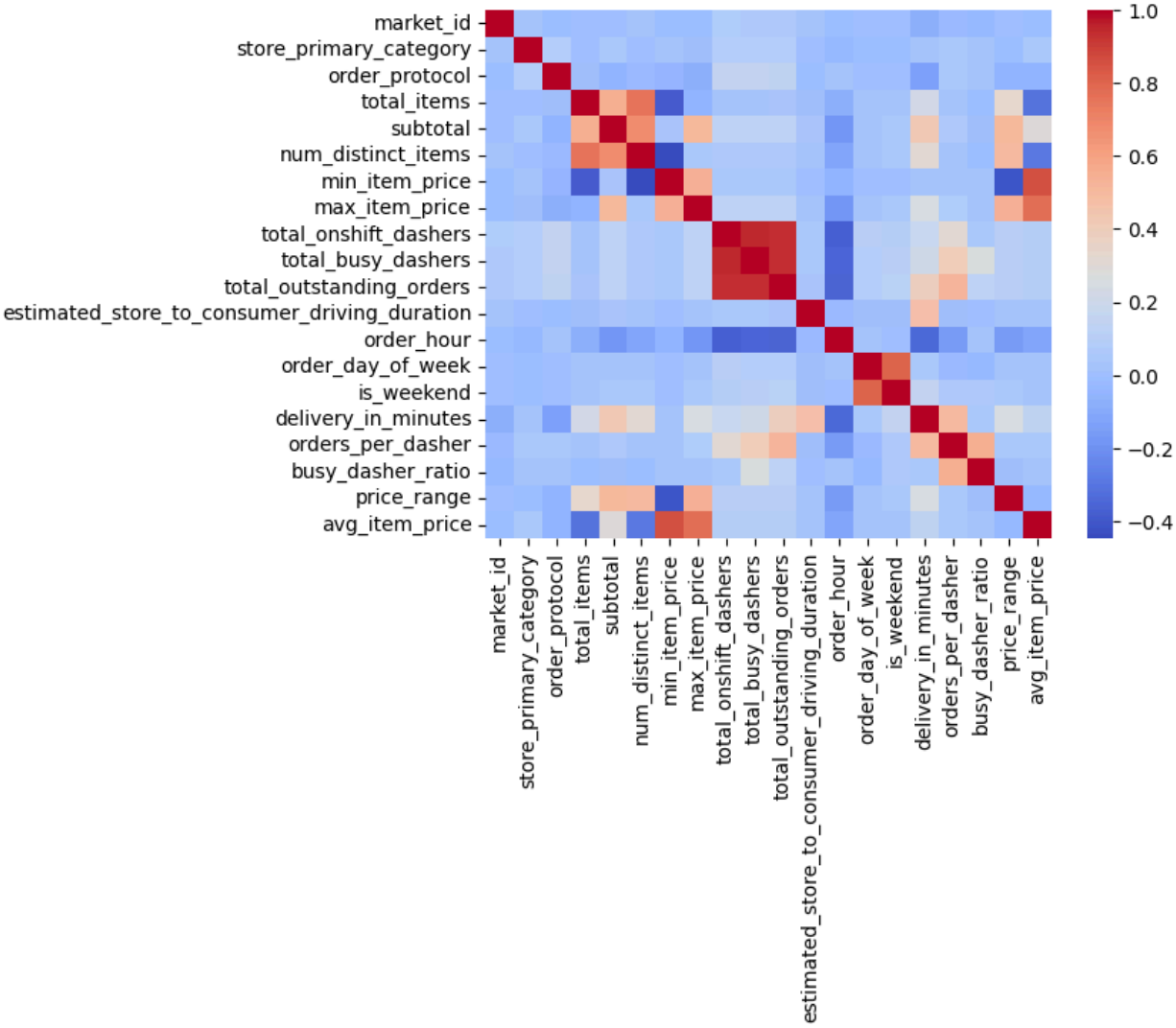
In [107…

```
corr = df_new.corr(numeric_only=True)

sns.heatmap(corr, cmap="coolwarm")
```

Out[107…

```
<Axes: >
```



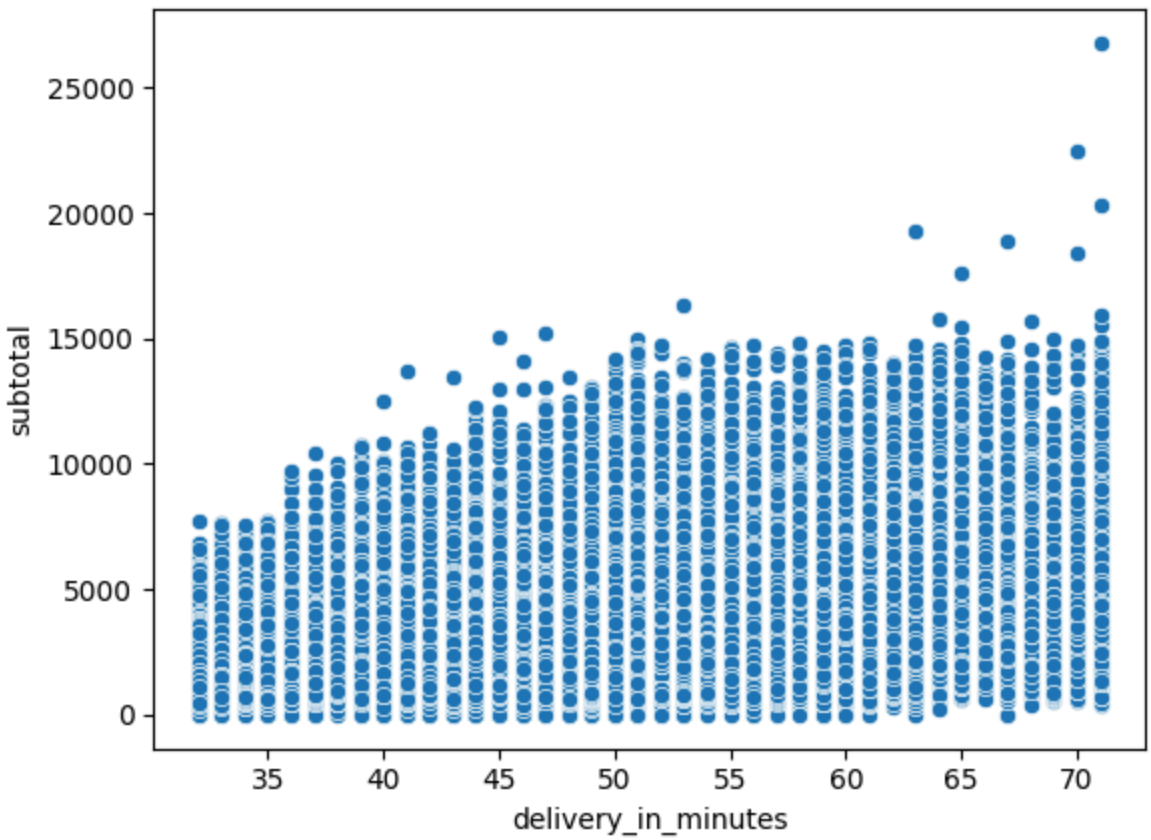
We can see from the correlation plot shown above

1. the features correlated with target variable delivery in minutes are subtotal,estimated_store_to_consumer_driving_duration, total_outstanding_orders etc
2. we can see that orders per dasher and total items are highly correlated to busy dasher ratio..which is quite evident from the fact that as number of orders per dasher increases , the dasher becomes busy more likely

Delivery Time vs Number of Items

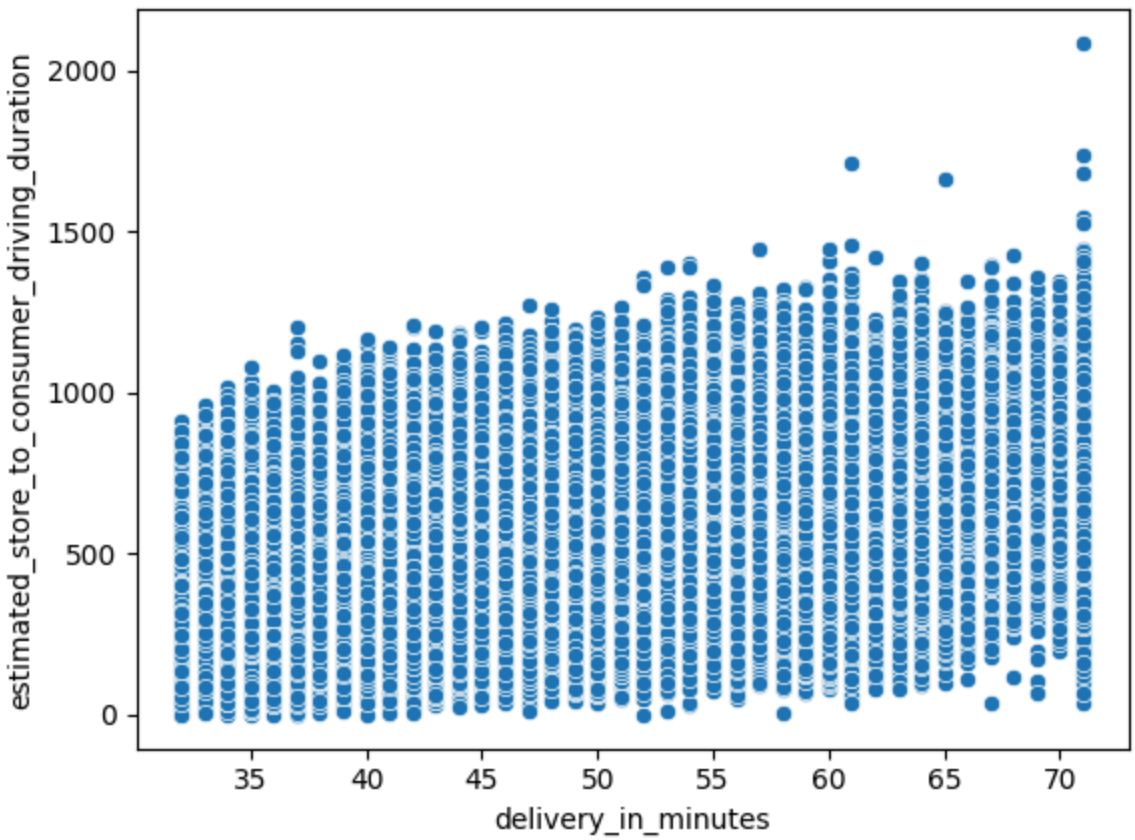
In [108…
```
sns.scatterplot(x="delivery_in_minutes", y = "subtotal", data= df_new)
```

Out[108…
```
<Axes: xlabel='delivery_in_minutes', ylabel='subtotal'>
```



In [109…
```
sns.scatterplot(x="delivery_in_minutes", y = "estimated_store_to_consumer_driv
```

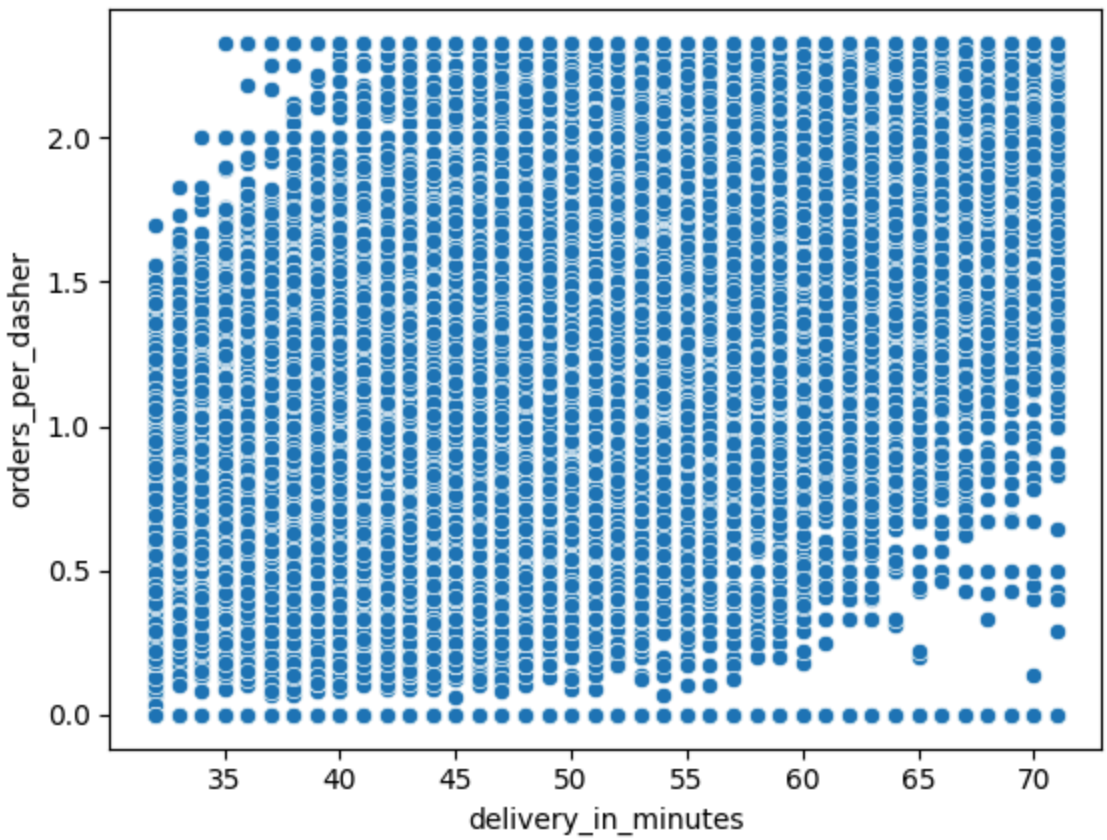
Out[109…
```
<Axes: xlabel='delivery_in_minutes', ylabel='estimated_store_to_consumer_driv
ing_duration'>
```



More distance → longer delivery time

In [110…
```
sns.scatterplot(x="delivery_in_minutes", y = "orders_per_dasher", data= df_new
```

Out[110…
```
<Axes: xlabel='delivery_in_minutes', ylabel='orders_per_dasher'>
```



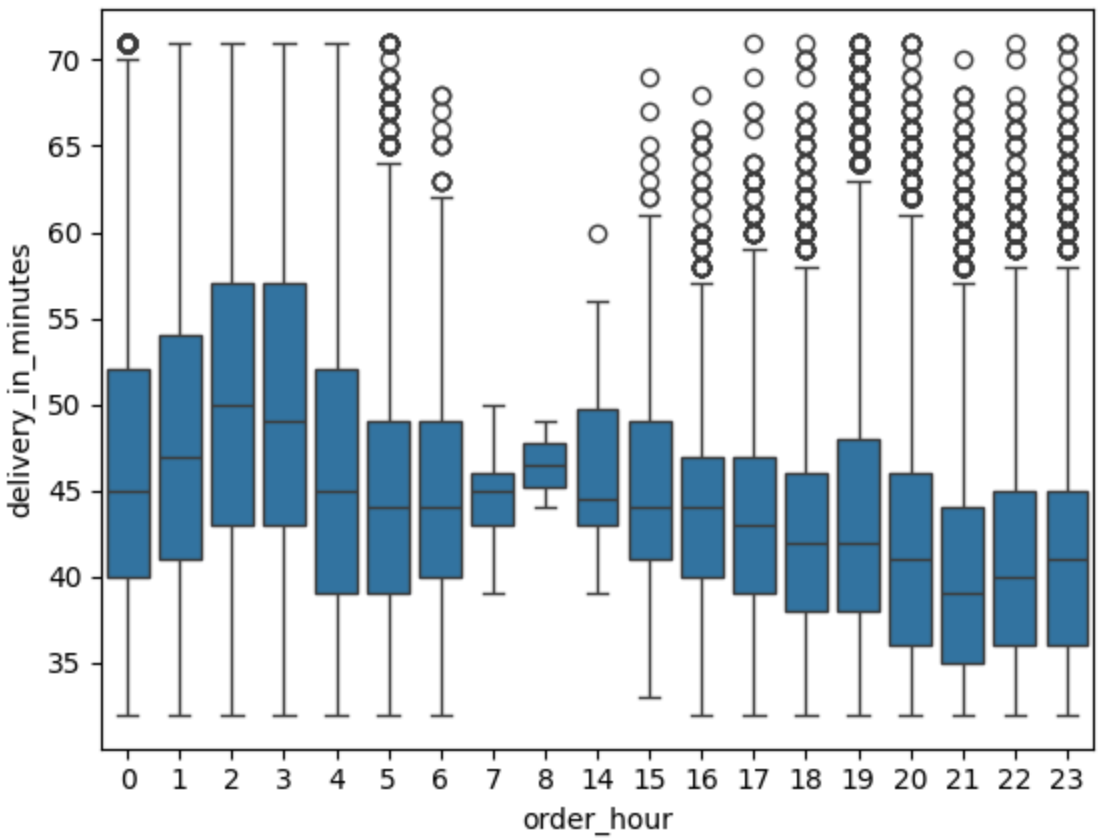
We can see a slight positive correlation between delivery in minutes and order per dasher.

In [111…

```
sns.boxplot(
    x="order_hour",
    y="delivery_in_minutes",
    data=df_new
)
```

Out[111…

```
<Axes: xlabel='order_hour', ylabel='delivery_in_minutes'>
```



Delivery delays are strongly influenced by time of day. during Evening (18–21 hrs) and Late Night / Early Morning (0–3 hrs, delivery times are the highest.

In [114…
```
df_new.head()
```

Out[114…

	market_id	created_at	actual_delivery_time	store_primary_category	order_pr
0	1.0	2015-02-06 22:24:17	2015-02-06 23:11:17	4	
1	2.0	2015-02-10 21:49:25	2015-02-10 22:33:25	46	
2	2.0	2015-02-16 00:11:35	2015-02-16 01:06:35	36	
3	1.0	2015-02-12 03:36:46	2015-02-12 04:35:46	38	
4	1.0	2015-01-27 02:12:36	2015-01-27 02:58:36	38	

5 rows × 23 columns

Data preparation for Random Forest

In [153…
```
df_new2= df_new.copy()
```

In [154…
```
df_new2= df_new2.drop(["created_at", "actual_delivery_time", "Time_taken_deliv
```

In [155…
```
rf_features = [
    "market_id",
    "store_primary_category",
    "order_protocol",
    "total_items",
    "num_distinct_items",
    "avg_item_price",
    "price_range",
    "orders_per_dasher",
    "busy_dasher_ratio",
    "estimated_store_to_consumer_driving_duration",
    "order_hour",
    "order_day_of_week",
    "is_weekend"
]
```

One-Hot Encode Categorical Variables

In [156…
```
cat_cols = ["market_id", "store_primary_category", "order_protocol"]
target = "delivery_in_minutes"
```

In [157…
```
X_rf = pd.get_dummies(
    df_new2[rf_features],
    columns=cat_cols,
    drop_first=True
)

y_rf = df_new2[target]
```

In [158…
```
X_train_rf, X_test_rf, y_train_rf, y_test_rf = train_test_split(
    X_rf,
    y_rf,
    test_size=0.2,
    random_state=42
)
```

Train Random Forest Model

In [160…
```
from sklearn.ensemble import RandomForestRegressor
```

```
rf_model = RandomForestRegressor(
    n_estimators=200,
    max_depth=None,
    min_samples_split=5,
    min_samples_leaf=2,
    n_jobs=-1,
    random_state=42
)

rf_model.fit(X_train_rf, y_train_rf)
```

Out[160…

```
RandomForestRegressor
RandomForestRegressor(min_samples_leaf=2, min_samples_split=5, n_estimators=200,
                      n_jobs=-1, random_state=42)
```

Generate Predictions

In [161…

```
y_pred_rf = rf_model.predict(X_test_rf)
```

Compute Error Metric

In [162…

```
mae_rf = mean_absolute_error(y_test_rf, y_pred_rf)
rmse_rf = np.sqrt(mean_squared_error(y_test_rf, y_pred_rf))
r2_rf = r2_score(y_test_rf, y_pred_rf)

print("Random Forest Results")
print("---------------------")
print("MAE :", mae_rf)
print("RMSE:", rmse_rf)
print("R2  :", r2_rf)
```

```
Random Forest Results
---------------------
MAE : 2.014172072062913
RMSE: 2.7251164740870037
R2  : 0.9107377715594683
```

Feature Importance

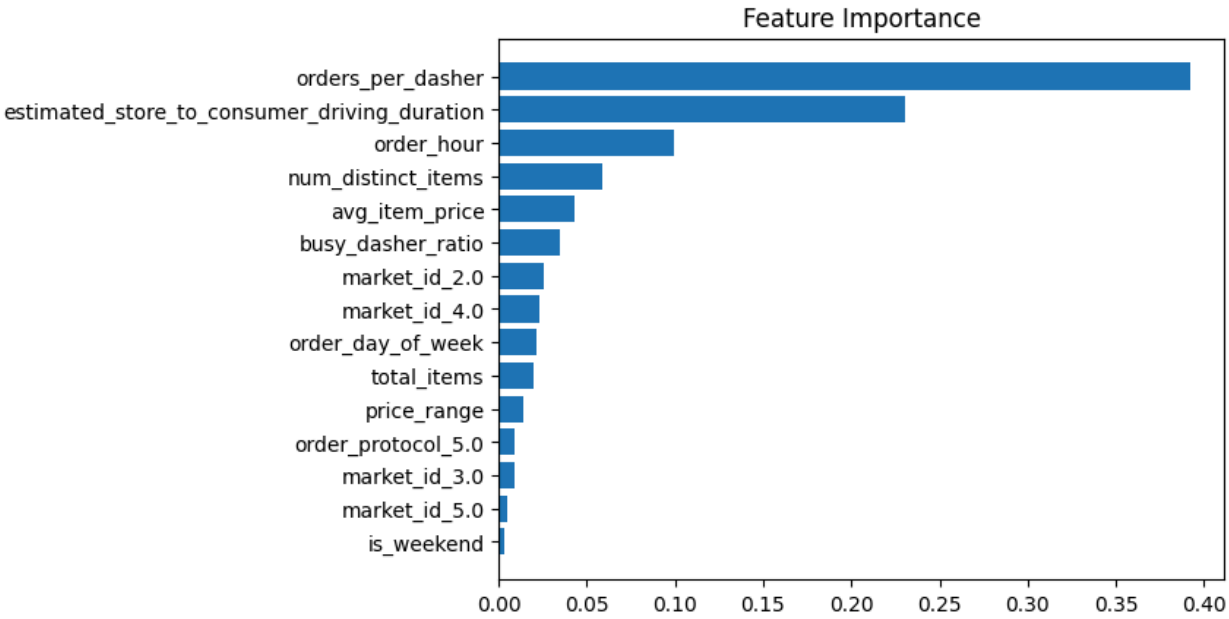
In [163…

```
importances = rf_model.feature_importances_
features = X_train_rf.columns

indices = np.argsort(importances)[-15:]

plt.barh(range(len(indices)), importances[indices])
```

```
plt.yticks(range(len(indices)), features[indices])
plt.title("Feature Importance")
plt.show()
```



The most important features for the model are :

1: orders_per_dasher

2: estimated_store_to_consumer_driving_duration

3: order_hour

4: number_of_distinct_items

5:busy_dasher_ratio

Data Preparation for NN Model Training

In [115…
```
df_model= df_new.copy()
```

In [116…
```
df_model= df_model.drop(["created_at", "actual_delivery_time", "Time_taken_del
```

In [117…
```
df_model.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 175777 entries, 0 to 175776
Data columns (total 14 columns):
 #   Column                                        Non-Null Count   Dtype
---  ------                                        --------------   -----
 0   market_id                                     175777 non-null  float64
 1   store_primary_category                        175777 non-null  int64
 2   order_protocol                                175777 non-null  float64
 3   total_items                                   175777 non-null  int64
 4   num_distinct_items                            175777 non-null  int64
 5   estimated_store_to_consumer_driving_duration  175777 non-null  float64
 6   order_hour                                    175777 non-null  int32
 7   order_day_of_week                             175777 non-null  int32
 8   is_weekend                                    175777 non-null  int64
 9   delivery_in_minutes                           175777 non-null  float64
 10  orders_per_dasher                             175777 non-null  float64
 11  busy_dasher_ratio                             175777 non-null  float64
 12  price_range                                   175777 non-null  int64
 13  avg_item_price                                175777 non-null  float64
dtypes: float64(7), int32(2), int64(5)
memory usage: 17.4 MB
```

Separate Categorical and Numeric Columns

In [120…
```
cat_cols = ["market_id", "store_primary_category", "order_protocol"]
```

In [121…
```
num_cols = [
    "total_items",
    "num_distinct_items",
    "avg_item_price",
    "price_range",
    "orders_per_dasher",
    "busy_dasher_ratio",
    "estimated_store_to_consumer_driving_duration",
    "order_hour",
    "order_day_of_week",
    "is_weekend"
]
```

In [122…
```
target = "delivery_in_minutes"
```

In [123…
```
X_cat = df_model[cat_cols]
X_num = df_model[num_cols]
y = df_model[target]
```

Train Test Split

In [124…
```
X_cat_train, X_cat_test, X_num_train, X_num_test, y_train, y_test = train_test
    X_cat, X_num, y, test_size=0.2, random_state=42)
```

Scale Only numerical features

In [125…
```
scaler = StandardScaler()

X_num_train = scaler.fit_transform(X_num_train)
X_num_test = scaler.transform(X_num_test)
```

In [135…
```
X_market_train = X_cat_train["market_id"].values
X_market_test = X_cat_test["market_id"].values

X_store_train = X_cat_train["store_primary_category"].values
X_store_test = X_cat_test["store_primary_category"].values

X_protocol_train = X_cat_train["order_protocol"].values
X_protocol_test = X_cat_test["order_protocol"].values
```

Build the Embedding Model

Inputs

In [126…
```
market_input = Input(shape=(1,))
store_input = Input(shape=(1,))
protocol_input = Input(shape=(1,))
numeric_input = Input(shape=(X_num_train.shape[1],))
```

To calculate the embedding dimension we will use this formula : min(50, category/2)

Embedding Layers

In [127…
```
market_emb = Embedding(input_dim=df_model["market_id"].nunique(), output_dim=3
market_emb = Flatten()(market_emb)

store_emb = Embedding(input_dim=df_model["store_primary_category"].nunique(),
store_emb = Flatten()(store_emb)

protocol_emb = Embedding(input_dim=df_model["order_protocol"].nunique(), outpu
protocol_emb = Flatten()(protocol_emb)
```

Combine Inputs

In [128…
```
x = Concatenate()([
    market_emb,
    store_emb,
    protocol_emb,
    numeric_input
])
```

Model Training NN

Training a Baseline NN Model

In [129…

```
x = BatchNormalization()(x)

x = Dense(128, activation="relu")(x)
x = BatchNormalization()(x)
x = Dropout(0.3)(x)

x = Dense(64, activation="relu")(x)
x = BatchNormalization()(x)
x = Dropout(0.2)(x)

x = Dense(32, activation="relu")(x)

output = Dense(1)(x)
```

In [130…

```
model = Model(
    inputs=[market_input, store_input, protocol_input, numeric_input],
    outputs=output
)
```

In [131…

```
optimizer = Adam(learning_rate=0.0005)
```

In [132…

```
early_stop = EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)
```

In [133…

```
lr_scheduler = ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.5,
    patience=4,
    min_lr=1e-5
)
```

In [136…

```
model.compile(optimizer= optimizer ,loss = "mse", metrics = ["mae"])
```

In [137…

```
history= model.fit([X_market_train, X_store_train, X_protocol_train, X_num_tra
```

```
Epoch 1/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - loss: 614.0791 - mae: 16.4676 - va
l_loss: 16.0809 - val_mae: 3.0082 - learning_rate: 5.0000e-04
Epoch 2/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 17.2655 - mae: 2.9064 - val_lo
ss: 12.6199 - val_mae: 2.6438 - learning_rate: 5.0000e-04
Epoch 3/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 13.3574 - mae: 2.6087 - val_lo
ss: 11.3873 - val_mae: 2.5099 - learning_rate: 5.0000e-04
Epoch 4/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 11.1438 - mae: 2.4624 - val_lo
ss: 10.3023 - val_mae: 2.3745 - learning_rate: 5.0000e-04
Epoch 5/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 10.0243 - mae: 2.3317 - val_lo
ss: 9.7129 - val_mae: 2.3263 - learning_rate: 5.0000e-04
Epoch 6/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 8.8873 - mae: 2.2034 - val_los
s: 8.3454 - val_mae: 2.1360 - learning_rate: 5.0000e-04
Epoch 7/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 8.1164 - mae: 2.1017 - val_los
s: 8.1200 - val_mae: 2.1229 - learning_rate: 5.0000e-04
Epoch 8/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - loss: 7.6886 - mae: 2.0519 - val_los
s: 7.7340 - val_mae: 2.0613 - learning_rate: 5.0000e-04
Epoch 9/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 7.3781 - mae: 2.0087 - val_los
s: 7.3281 - val_mae: 1.9942 - learning_rate: 5.0000e-04
Epoch 10/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 7.1455 - mae: 1.9713 - val_los
s: 7.1495 - val_mae: 1.9661 - learning_rate: 5.0000e-04
Epoch 11/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 7.0666 - mae: 1.9592 - val_los
s: 7.1294 - val_mae: 1.9728 - learning_rate: 5.0000e-04
Epoch 12/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 6.8632 - mae: 1.9284 - val_los
s: 7.2671 - val_mae: 1.9927 - learning_rate: 5.0000e-04
Epoch 13/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 6.7645 - mae: 1.9118 - val_los
s: 6.7975 - val_mae: 1.9092 - learning_rate: 5.0000e-04
Epoch 14/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 6.6607 - mae: 1.8963 - val_los
s: 6.6444 - val_mae: 1.8894 - learning_rate: 5.0000e-04
Epoch 15/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 6.4992 - mae: 1.8751 - val_los
s: 7.2023 - val_mae: 2.0038 - learning_rate: 5.0000e-04
Epoch 16/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 6.3489 - mae: 1.8494 - val_los
s: 6.5767 - val_mae: 1.8801 - learning_rate: 5.0000e-04
Epoch 17/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 6.2938 - mae: 1.8379 - val_los
s: 6.4758 - val_mae: 1.8632 - learning_rate: 5.0000e-04
Epoch 18/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 6.5189 - mae: 1.8515 - val_los
s: 6.4850 - val_mae: 1.8740 - learning_rate: 5.0000e-04
```

```
Epoch 19/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 6.1564 - mae: 1.8099 - val_los
s: 6.2658 - val_mae: 1.8272 - learning_rate: 5.0000e-04
Epoch 20/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - loss: 6.0740 - mae: 1.7968 - val_los
s: 6.2151 - val_mae: 1.8209 - learning_rate: 5.0000e-04
Epoch 21/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 5.9589 - mae: 1.7843 - val_los
s: 6.1446 - val_mae: 1.8102 - learning_rate: 5.0000e-04
Epoch 22/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 5.9690 - mae: 1.7828 - val_los
s: 6.1226 - val_mae: 1.7973 - learning_rate: 5.0000e-04
Epoch 23/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 7s 6ms/step - loss: 5.8643 - mae: 1.7658 - val_los
s: 6.5845 - val_mae: 1.8956 - learning_rate: 5.0000e-04
Epoch 24/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 7s 3ms/step - loss: 5.7266 - mae: 1.7440 - val_los
s: 6.1603 - val_mae: 1.8098 - learning_rate: 5.0000e-04
Epoch 25/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 5.7812 - mae: 1.7498 - val_los
s: 5.8753 - val_mae: 1.7434 - learning_rate: 5.0000e-04
Epoch 26/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 5.6332 - mae: 1.7273 - val_los
s: 6.0520 - val_mae: 1.7891 - learning_rate: 5.0000e-04
Epoch 27/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 5.8217 - mae: 1.7468 - val_los
s: 6.0696 - val_mae: 1.7987 - learning_rate: 5.0000e-04
Epoch 28/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 5.5432 - mae: 1.7104 - val_los
s: 5.7934 - val_mae: 1.7345 - learning_rate: 5.0000e-04
Epoch 29/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - loss: 5.5778 - mae: 1.7158 - val_los
s: 6.0027 - val_mae: 1.7833 - learning_rate: 5.0000e-04
Epoch 30/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 5.5616 - mae: 1.7116 - val_los
s: 5.9262 - val_mae: 1.7471 - learning_rate: 5.0000e-04
Epoch 31/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 5.4756 - mae: 1.6987 - val_los
s: 5.8145 - val_mae: 1.7405 - learning_rate: 5.0000e-04
Epoch 32/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 5.4136 - mae: 1.6878 - val_los
s: 5.6933 - val_mae: 1.7070 - learning_rate: 5.0000e-04
Epoch 33/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 5.4426 - mae: 1.6934 - val_los
s: 5.7827 - val_mae: 1.7428 - learning_rate: 5.0000e-04
Epoch 34/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - loss: 5.4421 - mae: 1.6909 - val_los
s: 5.5316 - val_mae: 1.6810 - learning_rate: 5.0000e-04
Epoch 35/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 5.3504 - mae: 1.6767 - val_los
s: 5.6183 - val_mae: 1.7049 - learning_rate: 5.0000e-04
Epoch 36/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 5.3443 - mae: 1.6732 - val_los
s: 5.5347 - val_mae: 1.6857 - learning_rate: 5.0000e-04
```

```
Epoch 37/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - loss: 5.2419 - mae: 1.6542 - val_los
s: 5.4992 - val_mae: 1.6822 - learning_rate: 5.0000e-04
Epoch 38/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 5.2891 - mae: 1.6680 - val_los
s: 5.5831 - val_mae: 1.6905 - learning_rate: 5.0000e-04
Epoch 39/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 5.2678 - mae: 1.6610 - val_los
s: 5.5025 - val_mae: 1.6831 - learning_rate: 5.0000e-04
Epoch 40/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 5.2156 - mae: 1.6497 - val_los
s: 5.5144 - val_mae: 1.6855 - learning_rate: 5.0000e-04
Epoch 41/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 5.1867 - mae: 1.6478 - val_los
s: 5.5353 - val_mae: 1.6918 - learning_rate: 5.0000e-04
Epoch 42/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 5.0771 - mae: 1.6148 - val_los
s: 5.4535 - val_mae: 1.6811 - learning_rate: 2.5000e-04
Epoch 43/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 5.0299 - mae: 1.6179 - val_los
s: 5.5040 - val_mae: 1.6890 - learning_rate: 2.5000e-04
Epoch 44/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 4.9838 - mae: 1.6077 - val_los
s: 5.4540 - val_mae: 1.6776 - learning_rate: 2.5000e-04
Epoch 45/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 5.0773 - mae: 1.6202 - val_los
s: 5.4576 - val_mae: 1.6754 - learning_rate: 2.5000e-04
Epoch 46/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 5.0059 - mae: 1.6109 - val_los
s: 5.5596 - val_mae: 1.6986 - learning_rate: 2.5000e-04
Epoch 47/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 4.9579 - mae: 1.5978 - val_los
s: 5.3018 - val_mae: 1.6433 - learning_rate: 1.2500e-04
Epoch 48/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 4.8900 - mae: 1.5878 - val_los
s: 5.3428 - val_mae: 1.6556 - learning_rate: 1.2500e-04
Epoch 49/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 4.8945 - mae: 1.5913 - val_los
s: 5.3614 - val_mae: 1.6603 - learning_rate: 1.2500e-04
Epoch 50/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - loss: 4.9154 - mae: 1.5914 - val_los
s: 5.2907 - val_mae: 1.6419 - learning_rate: 1.2500e-04
```

Evaluate the model

In [138…

```
model.evaluate(
    [X_market_test, X_store_test, X_protocol_test, X_num_test],
    y_test
)
```

```
1099/1099 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - loss: 5.3412 - mae: 1.6676
```

Out[138…

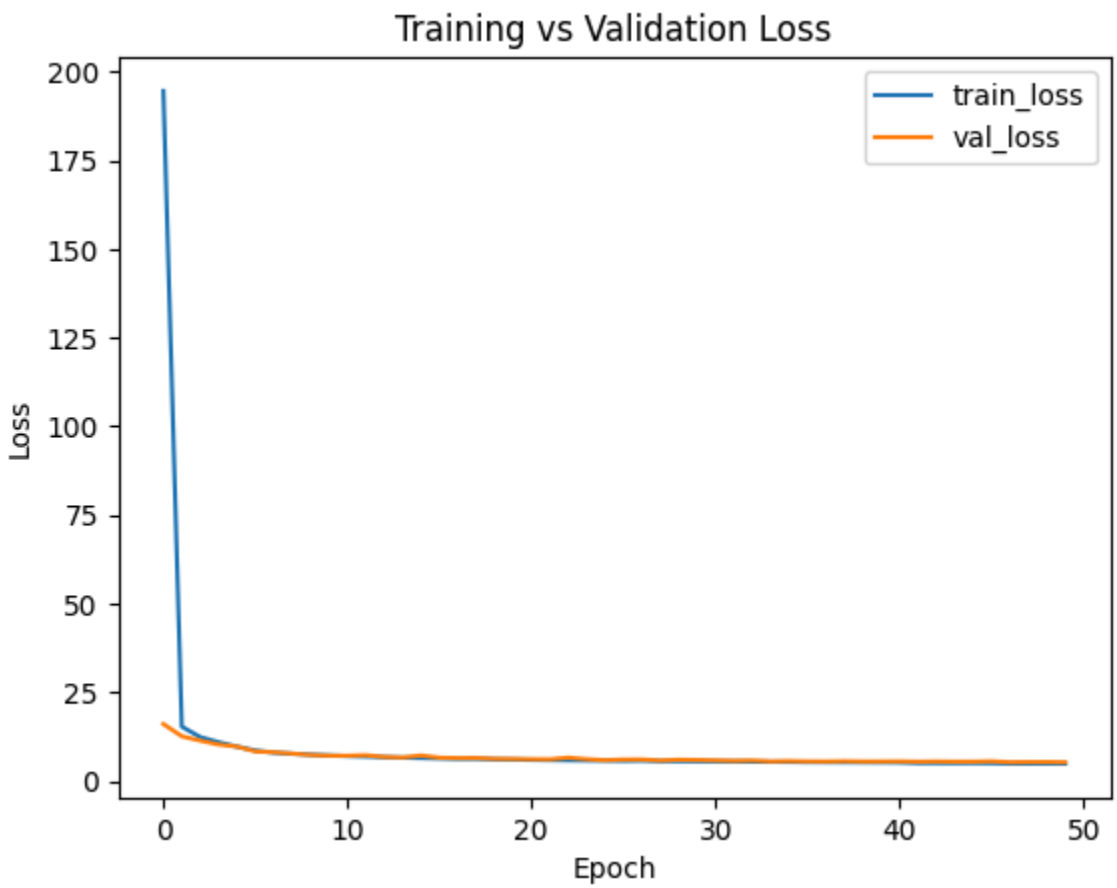
```
[5.259232521057129, 1.652323603630066]
```

Plot Loss vs Epoch

In [139…

```
plt.plot(history.history["loss"], label="train_loss")
plt.plot(history.history["val_loss"], label="val_loss")

plt.legend()
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")
plt.show()
```



Prediction

In [141…

```
y_pred = model.predict(
    [X_market_test, X_store_test, X_protocol_test, X_num_test]
)
```

```
1099/1099 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step
```

In [143…

```
y_pred = y_pred.flatten()
```

Compute Error Metric

In [145…
```
mae_nn = mean_absolute_error(y_test, y_pred)
rmse_nn = np.sqrt(mean_squared_error(y_test, y_pred))
r2_nn = r2_score(y_test, y_pred)
```

In [146…
```
print(f"MAE  : {mae_nn:.2f}")
print(f"RMSE : {rmse_nn:.2f}")
print(f"R2   : {r2_nn:.3f}")
```

```
MAE  : 1.65
RMSE : 2.29
R2   : 0.937
```

R^2 tells how much variance the model explains.In our case R squared is > 90% which explain model can explains most of the variance in the data.

RMSE is : 2.29 minutes

mae is 1.65 minutes.On average the model misses delivery time by ~1.65 minutes

Compare Random Forest vs Neural Network

In [165…
```
results = pd.DataFrame({
    "Model": ["Neural Network", "Random Forest"],
    "MAE": [mae, mae_rf],
    "RMSE": [rmse, rmse_rf],
    "R2": [r2, r2_rf]
})

print(results)
```

```
           Model       MAE      RMSE        R2
0  Neural Network  1.652323  2.293302  0.936785
1   Random Forest  2.014172  2.725116  0.910738
```

Here we can see that NN performs better than random Forest model which is evident from the fact that NN works well on large dataset. Usually for tabular data people prefer classical ML models but her we can see NN works very well.

Hyperparameter Tuning (Random Search) BY Keras Tuner

In [147…
```
import keras_tuner as kt
```

Define the Hyperparameter Model Function

In [148…

```
def build_model(hp):

    # Inputs
    market_input = Input(shape=(1,))
    store_input = Input(shape=(1,))
    protocol_input = Input(shape=(1,))
    numeric_input = Input(shape=(X_num_train.shape[1],))

    # Embeddings
    market_emb = Embedding(
        input_dim=df_model["market_id"].nunique(),
        output_dim=3
    )(market_input)

    market_emb = Flatten()(market_emb)

    store_emb = Embedding(
        input_dim=df_model["store_primary_category"].nunique(),
        output_dim=8
    )(store_input)

    store_emb = Flatten()(store_emb)

    protocol_emb = Embedding(
        input_dim=df_model["order_protocol"].nunique(),
        output_dim=2
    )(protocol_input)

    protocol_emb = Flatten()(protocol_emb)

    # Concatenate features
    x = Concatenate()([
        market_emb,
        store_emb,
        protocol_emb,
        numeric_input
    ])

    # Optional BatchNorm
    if hp.Boolean("batch_norm"):
        x = BatchNormalization()(x)

    # Tune number of hidden layers
    for i in range(hp.Int("num_layers", 1, 4)):

        units = hp.Int(
            f"units_{i}",
            min_value=32,
            max_value=256,
            step=32
```

```
        )

        activation = hp.Choice(
            "activation",
            ["relu", "elu", "tanh"]
        )

        x = Dense(units, activation=activation)(x)

        if hp.Boolean("batch_norm"):
            x = BatchNormalization()(x)

        if hp.Boolean("dropout"):
            x = Dropout(
                hp.Float(
                    "dropout_rate",
                    min_value=0.1,
                    max_value=0.5,
                    step=0.1
                )
            )(x)

    output = Dense(1)(x)

    model = Model(
        inputs=[market_input, store_input, protocol_input, numeric_input],
        outputs=output
    )

    # Tune learning rate
    lr = hp.Choice(
        "learning_rate",
        [1e-2, 1e-3, 5e-4, 1e-4]
    )

    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss="mse",
        metrics=["mae"]
    )

    return model
```

Create the Hyperparameter Tuner

In [151…

```
tuner = kt.RandomSearch(
    build_model,
    objective="val_loss",
    max_trials=5,
    executions_per_trial=1,
    directory="tuner_results",
    project_name="delivery_time_model"
```

```
)
```

```
Reloading Tuner from tuner_results/delivery_time_model/tuner0.json
```

max_trials = 20 different models tested

Run Hyperparameter Search

In [166…

```
tuner.search(
    [X_market_train, X_store_train, X_protocol_train, X_num_train],
    y_train,
    validation_split=0.2,
    epochs=30,
    batch_size=128
)
```

```
Trial 6 Complete [00h 08m 26s]
val_loss: 7.009742259979248

Best val_loss So Far: 4.861782073974609
Total elapsed time: 00h 21m 49s
```

In [ ]:

```

```

Retrieve Best Model

In [167…

```
best_model = tuner.get_best_models(num_models=1)[0]
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/saving/saving_lib.py:802: Use
rWarning: Skipping variable loading for optimizer 'adam', because it has 2 vari
ables whereas the saved optimizer has 40 variables.
  saveable.load_own_variables(weights_store.get(inner_path))
```

In [168…

```
best_hp = tuner.get_best_hyperparameters(1)[0]

print(best_hp.values)
```

```
{'batch_norm': True, 'num_layers': 3, 'units_0': 192, 'activation': 'elu', 'dro
pout': False, 'learning_rate': 0.01, 'dropout_rate': 0.30000000000000004, 'unit
s_1': 224, 'units_2': 64, 'units_3': 256}
```

Plot Loss Vs Epoch

In [170…

```
history = best_model.fit(
    [X_market_train, X_store_train, X_protocol_train, X_num_train],
    y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=128
)
```

```
Epoch 1/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 4.6662 - mae: 1.6242 - val_los
s: 5.3459 - val_mae: 1.6309
Epoch 2/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 4.7546 - mae: 1.6400 - val_los
s: 4.8760 - val_mae: 1.5272
Epoch 3/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 4.8984 - mae: 1.6636 - val_los
s: 4.8350 - val_mae: 1.4995
Epoch 4/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - loss: 4.7754 - mae: 1.6465 - val_los
s: 6.7088 - val_mae: 1.8918
Epoch 5/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - loss: 4.7887 - mae: 1.6509 - val_los
s: 4.9075 - val_mae: 1.5303
Epoch 6/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - loss: 4.6484 - mae: 1.6185 - val_los
s: 5.8984 - val_mae: 1.7096
Epoch 7/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - loss: 4.5905 - mae: 1.6119 - val_los
s: 4.8212 - val_mae: 1.5030
Epoch 8/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - loss: 4.6958 - mae: 1.6351 - val_los
s: 5.0275 - val_mae: 1.5996
Epoch 9/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 7s 7ms/step - loss: 4.7037 - mae: 1.6324 - val_los
s: 5.2837 - val_mae: 1.6377
Epoch 10/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - loss: 4.8320 - mae: 1.6559 - val_los
s: 5.4675 - val_mae: 1.6754
Epoch 11/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - loss: 4.7646 - mae: 1.6433 - val_los
s: 5.0812 - val_mae: 1.5923
Epoch 12/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 4.6505 - mae: 1.6198 - val_los
s: 4.8321 - val_mae: 1.5160
Epoch 13/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 6s 7ms/step - loss: 4.6282 - mae: 1.6215 - val_los
s: 4.9502 - val_mae: 1.5453
Epoch 14/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - loss: 4.6565 - mae: 1.6275 - val_los
s: 5.5765 - val_mae: 1.6787
Epoch 15/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 4.8167 - mae: 1.6638 - val_los
s: 5.9851 - val_mae: 1.5799
Epoch 16/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - loss: 4.6915 - mae: 1.6309 - val_los
s: 5.6788 - val_mae: 1.6861
Epoch 17/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 4.5495 - mae: 1.6072 - val_los
s: 5.5002 - val_mae: 1.6639
Epoch 18/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 4.6042 - mae: 1.6207 - val_los
s: 5.0885 - val_mae: 1.5380
```

```
Epoch 19/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 4.6233 - mae: 1.6191 - val_los
s: 5.1615 - val_mae: 1.6100
Epoch 20/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - loss: 4.6266 - mae: 1.6245 - val_los
s: 5.5326 - val_mae: 1.6908
Epoch 21/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - loss: 4.6775 - mae: 1.6357 - val_los
s: 5.4823 - val_mae: 1.6209
Epoch 22/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 4.6328 - mae: 1.6198 - val_los
s: 5.0163 - val_mae: 1.5485
Epoch 23/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - loss: 4.4625 - mae: 1.5932 - val_los
s: 5.2331 - val_mae: 1.6109
Epoch 24/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 4.6092 - mae: 1.6228 - val_los
s: 5.8636 - val_mae: 1.7450
Epoch 25/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - loss: 4.6654 - mae: 1.6303 - val_los
s: 6.7492 - val_mae: 2.0181
Epoch 26/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 6s 7ms/step - loss: 4.7163 - mae: 1.6454 - val_los
s: 4.9533 - val_mae: 1.5292
Epoch 27/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 4.6182 - mae: 1.6263 - val_los
s: 5.9980 - val_mae: 1.8058
Epoch 28/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 4.5323 - mae: 1.6092 - val_los
s: 5.8279 - val_mae: 1.7357
Epoch 29/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - loss: 4.6322 - mae: 1.6279 - val_los
s: 5.0627 - val_mae: 1.6141
Epoch 30/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - loss: 4.4844 - mae: 1.6014 - val_los
s: 5.9655 - val_mae: 1.7583
Epoch 31/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 4.4181 - mae: 1.5886 - val_los
s: 6.2449 - val_mae: 1.8548
Epoch 32/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - loss: 4.5247 - mae: 1.6071 - val_los
s: 5.4871 - val_mae: 1.6836
Epoch 33/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 6s 6ms/step - loss: 4.4158 - mae: 1.5808 - val_los
s: 5.2670 - val_mae: 1.6302
Epoch 34/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 6s 7ms/step - loss: 4.7398 - mae: 1.6492 - val_los
s: 5.5346 - val_mae: 1.6685
Epoch 35/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - loss: 4.3984 - mae: 1.5801 - val_los
s: 4.9334 - val_mae: 1.5402
Epoch 36/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 4.4750 - mae: 1.6035 - val_los
s: 4.8505 - val_mae: 1.5299
```

```
Epoch 37/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - loss: 4.7014 - mae: 1.6430 - val_los
s: 5.5692 - val_mae: 1.6639
Epoch 38/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 4.5729 - mae: 1.6150 - val_los
s: 5.3028 - val_mae: 1.6117
Epoch 39/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 4.3602 - mae: 1.5765 - val_los
s: 4.9078 - val_mae: 1.5237
Epoch 40/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 4.3601 - mae: 1.5768 - val_los
s: 5.8495 - val_mae: 1.7546
Epoch 41/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - loss: 4.4487 - mae: 1.5961 - val_los
s: 4.9177 - val_mae: 1.5520
Epoch 42/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 4.5611 - mae: 1.6126 - val_los
s: 5.0990 - val_mae: 1.5634
Epoch 43/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 4.5950 - mae: 1.6243 - val_los
s: 4.8225 - val_mae: 1.5006
Epoch 44/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 4.4209 - mae: 1.5923 - val_los
s: 5.2803 - val_mae: 1.5375
Epoch 45/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - loss: 4.3551 - mae: 1.5772 - val_los
s: 5.7155 - val_mae: 1.6567
Epoch 46/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 4.4385 - mae: 1.5956 - val_los
s: 4.8786 - val_mae: 1.5153
Epoch 47/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 4.4293 - mae: 1.5938 - val_los
s: 7.5776 - val_mae: 2.1100
Epoch 48/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - loss: 4.4179 - mae: 1.5929 - val_los
s: 5.4759 - val_mae: 1.6921
Epoch 49/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 6s 7ms/step - loss: 4.5399 - mae: 1.6269 - val_los
s: 5.0589 - val_mae: 1.5361
Epoch 50/50
879/879 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - loss: 4.6454 - mae: 1.6366 - val_los
s: 5.6570 - val_mae: 1.6962
```

In [171…

```
best_model.evaluate(
    [X_market_test, X_store_test, X_protocol_test, X_num_test],
    y_test
)
```

```
1099/1099 ━━━━━━━━━━━━━━━━━━━━ 7s 6ms/step - loss: 5.5962 - mae: 1.6992
```

Out[171…

```
[5.5870041847229, 1.69205641746521]
```

Predictions

In [172…

```
y_pred = best_model.predict(
    [X_market_test, X_store_test, X_protocol_test, X_num_test]
)

y_pred = y_pred.flatten()
```

```
1099/1099 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step
```

Compute the Error Metric

In [173…

```
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("RMSE:", rmse)
print("R2:", r2)
```

```
MAE: 1.6920549893680321
RMSE: 2.363684757964897
R2: 0.9328452563063188
```

Leading Questions :

List 3 functions the pandas datetime provides with one line explanation.

Ans:

.dt.hour :Extracts the hour (0–23) from a datetime column.

.dt.weekday : Returns the day of the week as an integer (0 = Monday, 6 = Sunday).

.dt.date : Extracts only the date part (YYYY-MM-DD) from a datetime value, removing the time component.

Short note on datetime, timedelta, time span (period)

Why do we need to check for outliers in our data?

Ans: Prevent model distortion, Improve model accuracy and stability, Detect data quality issues, Understand rare but important events

Name 3 outlier removal methods?

Ans:

1⃣ IQR Method (Interquartile Range) Detects outliers using the spread of the middle 50% of the data. Values outside Q1 − 1.5 × IQR or Q3 + 1.5 × IQR are considered outliers.

2⃣ Z-Score Method Measures how many standard deviations a value is from the mean. Values with |z-score| > 3 are usually treated as outliers.

3⃣ Percentile Clipping (Winsorization) Caps extreme values at certain percentiles (e.g., 1st and 99th percentile) instead of removing rows. Commonly used in ML to keep dataset size stable.

What classical machine learning methods can we use for this problem?

Random forest, Gradient Boosting

Why is scaling required for neural networks?

Ans

Ensures Stable Gradient Descent

Prevents Certain Features from Dominating

Improves Convergence Speed

Briefly explain your choice of optimizer.

Ans:

I chose the Adam optimizer because it combines the advantages of momentum and adaptive learning rates, making training faster and more stable.

Which activation function did you use and why?

Ans I used the ReLU (Rectified Linear Unit) activation function because it helps neural networks train faster and reduces common issues like the vanishing gradient problem.

ReLU outputs the input directly if it is positive and returns 0 for negative values. This simple behavior allows gradients to propagate efficiently through the network, enabling faster learning and better performance for deep models.

Why does a neural network perform well on a large dataset?

Ans

High Model Capacity : Neural networks contain many weights and layers

Reduced Overfitting

Better Representation Learning